Multi-Agent Portfolio Construction System

Session 1 — Deterministic Core: Complete Technical Documentation

Aidan Altorelli | Fordham MSQF Capstone | June 2026

This document covers every file built in Session 1: what each module does, why the design choices were made, the mathematical foundations behind each function, and how the pieces connect. It is intended as a complete reference that could be handed to a new engineer or a thesis committee member.

1. System Overview

The system is a **multi-agent LLM pipeline** for personalized portfolio construction that treats human capital as an asset on the investor's total economic balance sheet. The three-agent pipeline is:

- **Allocation Agent** — builds Black-Litterman optimized weights
- **Risk Agent** — evaluates those weights against deterministic thresholds
- **Compliance Agent** — final sign-off (not yet built)

Core architectural rule: the LLM is a pure interface.

Every number the system produces is computed in code inside `core/`. The LLM's only permitted jobs are: (1) validate inputs, (2) route data to the right function, (3) render the deterministic output as readable English. It never invents or modifies a number. This is what makes the system fiduciary-defensible and auditable.

File layout built in this session:

File / Folder	Purpose
portfolio_system/schemas.py	Pydantic data contracts for every agent handoff
portfolio_system/data/loaders.py	WRDS pulls for CRSP, Compustat, Fama-French
core/human_capital.py	JMN (1992) human-capital math — shared by both agents
core/constraints.py	Single source of truth for all four concentration limits
core/allocation.py	Black-Litterman optimizer + equilibrium returns
core/risk.py	Rule engine: VaR/CVaR, drawdown, stress, decision logic
tests/test_human_capital.py	Unit tests for human_capital.py (17 tests)
tests/test_constraints.py	Unit tests for constraints.py (16 tests)
tests/test_allocation.py	Unit tests for allocation.py (17 tests)
tests/test_risk.py	Unit tests for risk.py (29 tests)
pyproject.toml	Makes the project pip-installable; editable install active

2. schemas.py — Pydantic Data Contracts

`schemas.py` is the single file every other module imports types from. It defines what data looks like at every boundary in the system: user inputs, allocation outputs, risk outputs, and the FLAG-loop feedback. Using Pydantic means bad data is rejected at the boundary, not silently propagated into core math.

Enums

`RiskProfile` (Conservative / Moderate / Aggressive) — maps to drawdown caps and risk-aversion coefficients used throughout the system. `RiskDecision` (APPROVE / FLAG / REJECT) — the three outcomes of the Risk Agent. `StressSeverity` (LOW / MEDIUM / HIGH / CRITICAL) — grades how badly a stress scenario hurts relative to the investor's own drawdown cap. `ConstraintType` — labels what kind of concentration rule was violated, so the FLAG feedback loop knows how to tighten the optimizer.

User Inputs

Class	What it holds
HumanCapitalInput	PV of future labor income, employer ticker/sector, income volatility, income beta (correlation with mar
UserProfile	Financial wealth W, the HumanCapitalInput, and risk profile
InstrumentUniverse	Pre-approved ticker list, their market-cap weights (for BL prior), and GICS sector mapping
AllocationInput	UserProfile + universe + flag_constraints (empty on first run, populated on FLAG re-entry) + flag_itera

Allocation Output (→ Risk Agent)

Class	What it holds
WeightDecomposition	Per-ticker total_weight, equilibrium_baseline, view_tilt, human_capital_offset — breaks each weight i
FactorExposures	market_beta, smb, hml, mom — Fama-French + momentum betas from OLS regression
PortfolioStatistics	expected_return, volatility, sharpe_ratio, factor_exposures — all computed in core/
AllocationOutput	The full handoff: weights list (validated to sum to 1.0), risky_weight, safe_weight, portfolio stats, ration

Why the two model_validators on AllocationOutput?

The weights-sum-to-one check is a mathematical invariant, not a business rule. Pydantic validators run on every construction, so if a bug in `core/allocation.py` produces weights that sum to 0.99, the schema catches it immediately with a clear error rather than silently propagating a wrong portfolio through the rest of the pipeline.

Risk Output (→ Compliance)

Class	What it holds
VaRMetrics	var_95, var_99, cvar_95, cvar_99 — daily VaR and CVaR at both Basel confidence levels
ConcentrationFlags	Lists of breached tickers/sectors and a boolean for employer breach — populated by constraints.py

StressResult	One row per scenario: name, portfolio_loss, threshold_breached, severity
HumanCapitalMetrics	The HC team's full view: total_wealth, hc_fraction, effective equity exposure, economic sector exp
RiskMetrics	All metrics in one object: vol, VaR/CVaR, MDD, factor exposures, liquidity, concentration flags, HC m
RiskOutput	Decision + AllocationOutput + RiskMetrics + reasoning_trace + constraints_violated + flag_iteration
AllocationConstraint	One violated constraint: type, target (ticker or sector), current_value, limit — the feedback the Risk Ag

3. data/loaders.py — WRDS Data Pulls

This is the only file in the system that touches a database. Every function takes a `wrds.Connection` and returns a clean `pd.DataFrame` or `dict`. No math happens here — that separation means every core function is unit-testable with hand-built fixtures, no database access required in CI.

Function	WRDS Table	Returns	Used by
get_connection()	wrds.Connection()	Open WRDS connection	Agent layer
load_permno_map(tickers)	crsp.dsenames	DataFrame: ticker → PERMNO	All CRSP callers
ticker_to_name(tickers)	crsp.dsenames	dict{ticker: name}	Convenience wrapper
load_crsp_daily(...)	crsp.dsf	Daily returns, price, volume	core/risk.py VaR + stress
load_crsp_monthly(...)	crsp.msf	Monthly returns, mktcap	core/allocation.py cov matrix
load_risk_free_rate(...)	ff.factors_monthly	pd.Series: date → rf	HC discount rate, BL prior
load_gics_sectors()	crsp.stocknames	dict{ticker: sector}	concentration checks
load_fama_french_factors(...)	ff.factors_monthly	DataFrame: mktrf, smb, hml, umd, rf	BL views + factor regression
load_market_cap_weights(...)	crsp.msf	dict{permno: weight}	BL equilibrium prior

Key implementation details

PERMNO mapping: CRSP uses its own integer key (PERMNO), not ticker symbols. Every call to a CRSP table must first map tickers to PERMNOs via `crsp.dsenames`. The query filters by `namedt` and `nameendt` to get the mapping as-of a specific date — tickers change over time.

ABS(prc): CRSP stores price as negative when the quote is a bid-ask midpoint rather than an actual trade price. Every price pull uses `ABS(prc)` to normalize this quirk.

Risk-free rate source: The spec names 'CRSP Treasury files' but the implementation uses {C('ff.factors_monthly.rf')} which is the standard academic source and essentially identical to 30-day T-bill yields.

4. core/human_capital.py — BMS Framework

This module implements the Bodie-Merton-Samuelson (1992) human capital framework extended by Campbell & Viceira (2002). It is the shared mathematical engine used by both agents: the Allocation Agent uses it to compute the optimal risky asset weight, and the Risk Agent uses it to build the total economic balance sheet.

Mathematical foundations

	Source	What it computes
	Merton (1971)	Optimal Merton risky share without human capital
	BMS (1992)	Optimal financial risky share when HC is risk-free
H/W)·β_h	Campbell & Viceira (2002)	Extended to risky HC: income beta hedges away equity de
	BMS balance sheet	Financial wealth plus present value of human capital
H)	BMS balance sheet	Human capital as share of total economic wealth

Function-by-function explanation

merton_risky_share(expected_excess_return, portfolio_volatility, risk_profile)

Implements the Merton (1971) formula for the optimal fraction of financial wealth to hold in the risky portfolio, ignoring human capital. Risk aversion γ is mapped from `RiskProfile`: Conservative $\rightarrow$ 5.0, Moderate $\rightarrow$ 3.0, Aggressive $\rightarrow$ 2.0. These are standard academic calibrations. The result is capped at 1.0 — no leverage in the base case. This value becomes the input α to the HC-adjusted formulas below.

compute_w_fin(alpha, human_capital_pv, financial_wealth, income_beta)

The central function of the BMS framework. When income_beta = 0 (labor income is like a bond — certain, uncorrelated with the market), the investor can afford to take more financial risk because their HC already acts as a safe asset: w_fin = $\alpha \cdot (1 + H/W)$. When income_beta > 0 (e.g., a tech worker whose salary correlates with the NASDAQ), the HC already provides implicit equity exposure, so the optimal financial risky share is reduced by the hedge term $(H/W) \cdot \beta_h$. The result is clipped to [0, 1]: no short-selling the risky portfolio, no leverage.

employer_concentration(employer_financial_weight, financial_wealth, human_capital_pv)

Computes total economic exposure to the employer as a fraction of total wealth. Human capital is treated as 100% employer exposure — this is the conservative, fiduciary-defensible assumption. The 15% limit from the spec is checked in constraints.py against this output. For a tech executive with $500k financial wealth and $1M human capital, even holding zero employer stock gives 67% employer concentration — well above the 15% limit, which is exactly the structurally-unfixable REJECT case in risk.py.

economic_sector_exposures(financial_weights, sectors, financial_wealth, human_capital_pv, employer_sector)

Builds the total economic balance sheet view of sector exposures. For each ticker, its financial weight × financial_wealth is tallied by GICS sector. Then human capital (the full HC present value) is added to the employer's sector, because labor income is economically equivalent to a long position in that sector. The result is

expressed as fractions of total wealth (W + H) and sums to 1.0. This is the input to the economic sector concentration check (25% limit).

5. core/constraints.py — Concentration Limits

This is the single source of truth for all four concentration limits. The spec explicitly requires this: the same numbers must be used as optimizer bounds in `allocation.py` and as evaluation checks in `risk.py`. Having them in one file means you change a limit in one place and both agents automatically use the new value.

	Source	How enforced
	Russell Investments, CFA Institute	Optimizer upper
	HDFC TRU, industry practice	Optimizer linear
nt>	Internal	Risk check only
	Internal	Risk check only

Function-by-function explanation

aggregate_sector_weights(weights, sectors)

Simple utility that sums instrument weights by GICS sector. Used internally by check_sector and also imported by allocation.py when building the sector linear constraints for scipy.

check_single_name / check_sector / check_economic_sector / check_employer

Each function runs one concentration check and returns a list of `AllocationConstraint` objects — one per violation. The list is empty when the check passes. All comparisons use strict greater-than (>), not greater-than-or-equal, because a portfolio sitting exactly at the limit is compliant. This is the correct interpretation for all four limits.

run_all_checks(weights, sectors, economic_sector_exposures, employer_concentration)

Runs all four checks and aggregates the results into a `ConcentrationFlags` object. This is what goes into RiskMetrics — it's the reporting view. The lists of breached names are for human-readable output (reasoning trace).

all_violations(...)

Also runs all four checks but returns a flat `list[AllocationConstraint]` rather than a ConcentrationFlags object. This is the machine-readable view used to populate RiskOutput.constraints_violated on FLAG decisions, which then flows into AllocationInput.flag_constraints on re-entry.

6. core/allocation.py — Black-Litterman Optimizer

This is the most mathematically complex module. It implements the Black-Litterman (1990) model, which combines a CAPM-derived equilibrium prior with Fama-French factor views to produce posterior expected returns, then optimizes portfolio weights subject to concentration constraints.

The Black-Litterman pipeline

	Formula	Function
	Σ = cov(excess_returns) × 12	build_covariance_matrix()</f
	Π = δ · Σ · w_mkt (δ = 2.5)	compute_equilibrium_returns
	OLS: r_i = α_i + B_i·F + ε_i → Q, P, Ω	build_ff_views()
	μ_BL = A■¹[(τΣ)■¹Π + P'Ω■¹Q] (τ=0.025)	black_litterman()
ance	Σ_BL = Σ + A■¹	black_litterman()
	max w'μ – (δ/2)·w'Σw s.t. constraints	optimize_weights()
	w_fin from compute_w_fin(); risky/safe split	run_allocation()
	OLS: r_p = α + β·F	<font name='Courier' size='8'>compute_factor_exposures()

Why Black-Litterman instead of mean-variance?

Vanilla mean-variance optimization (Markowitz) is notorious for producing 'corner solutions' — portfolios with extreme concentration in a few assets and zero weight in many others. These solutions are extremely sensitive to small estimation errors in expected returns. Black-Litterman solves this by anchoring the prior to the CAPM equilibrium (which is diversified by construction) and only tilting away from it to the extent that the investor's views are confident. The result is a stable, diversified portfolio that is also fiduciary-defensible.

Why tau = 0.025?

τ controls how much weight the model places on the equilibrium prior vs the investor's views. τ = 0.025 is the value recommended by Walters (2013) based on empirical calibration. A small τ means high confidence in the equilibrium prior — conservative, appropriate for a fiduciary system.

How Fama-French views are constructed

For each asset i, OLS is run: r_i = α_i + B_i · F + ε_i, where F contains the four FF factors (MktRf, SMB, HML, UMD). The view Q_i is the factor-model-implied expected return: α_i·12 + B_i·λ_F where λ_F are the annualized mean factor premiums. P = I_N (one absolute view per asset). Ω = diag(annualized residual variances) / τ — larger residual variance means less confidence in that stock's view.

Optimizer constraints and FLAG re-entry

The optimizer uses scipy SLSQP with two types of constraints: (1) box bounds — 0 ≤ w_i ≤ 10% per ticker, (2) linear inequality constraints — one per GICS sector capping combined sector weight at 20%. On FLAG re-entry, violated constraints arrive in flag_constraints. SINGLE_NAME violations tighten that ticker's upper bound to 99% of the limit. SECTOR violations tighten the sector cap. EMPLOYER violations are translated to a max financial

weight for the employer ticker via _max_employer_financial_weight(). ECONOMIC_SECTOR violations scale down the financial sector limit proportional to the financial/total wealth ratio.

Weight decomposition

The spec requires each weight to be broken into three sources. equilibrium_baseline is the weight from optimizing with the CAPM prior Π only (views set to zero by inflating Ω). view_tilt is the shift caused by the FF views: w_BL – w_eq. human_capital_offset is the shift caused by FLAG constraints from the previous Risk Agent run: w_BL_constrained – w_BL_unconstrained. On the first run with no FLAG constraints all three of these terms sum to the final weight and the HC offset is zero.

7. core/risk.py — Deterministic Rule Engine

The Risk Agent evaluates a portfolio produced by the Allocation Agent against a fixed set of deterministic rules. It never modifies weights. Its only output is a decision (APPROVE / FLAG / REJECT) plus the metrics and reasoning trace that support that decision.

Risk measures

	Method	Standard
	Historical simulation: 5th percentile of daily portfolio P&L over last 5 years (1,260 trading days)	Basel III/IV
	Historical simulation: 1st percentile	Basel III/IV
	Mean of portfolio returns below the 5th percentile (Expected Shortfall)	Basel III/IV
	Mean of portfolio returns below the 1st percentile	Basel III/IV
	Maximum peak-to-trough loss of cumulative portfolio return	Uniform Prudent Investor Act

Why CVaR in addition to VaR?

VaR tells you the loss at the threshold percentile. CVaR (also called Expected Shortfall) tells you the average loss in the worst scenarios — the tail of the distribution. CVaR is a coherent risk measure (it satisfies subadditivity), which VaR is not. Basel III/IV requires both. For fat-tailed return distributions (which financial returns are), CVaR is the more informative measure.

Stress scenarios

	Benchmark loss	Method
-09	54%	Actual CRSP dai
-23	34%	Actual CRSP dai
-31	23.7%	Actual CRSP dai
	Computed	50% employer st

For each historical scenario, the function pulls actual CRSP daily returns for the portfolio's holdings during that date window, compounds them, and computes the cumulative loss. If CRSP data doesn't cover the period (e.g., a recent IPO), the function falls back to a beta-scaled approximation: portfolio_loss ≈ beta × benchmark_loss. The employer shock is synthetic: it models a 50% drop in employer stock × financial weight in that stock, plus a 30% hit to HC × HC fraction of total wealth.

Drawdown caps by risk profile

	Cap	If breached
	15%	FLAG: tighten all per-ticker limits proportionally
	20%	FLAG: tighten all per-ticker limits proportionally

	25%	FLAG: tighten all per-ticker limits proportionally

Stress severity thresholds

Severity is assigned relative to the investor's own drawdown cap (not absolute thresholds), so a 20% portfolio loss is CRITICAL for a conservative investor (cap = 15%) but only HIGH for an aggressive one (cap = 25%). LOW: loss < 50% of cap. MEDIUM: 50–100% of cap. HIGH: 100–150%. CRITICAL: >150%.

Decision logic: APPROVE / FLAG / REJECT

REJECT conditions (issue is upstream of Allocation — not fixable by reoptimizing):

1. flag_iteration ≥ 3 — three FLAG loops exhausted with no clean solution.
2. hc_fraction > EMPLOYER_LIMIT (15%) — human capital alone exceeds the employer concentration limit. Even if the investor holds zero employer stock, they still breach the limit. No portfolio reallocation can fix this.
3. CRITICAL stress result with no concentration violations and no drawdown breach — the overall risk level is just too high for any feasible portfolio.

FLAG conditions (fixable by sending constraints back to Allocation):

Any concentration violation (from constraints.py) or drawdown exceeding the cap. The violated constraints are packaged as AllocationConstraint objects and returned in RiskOutput.constraints_violated, which flows into AllocationInput.flag_constraints on the next iteration.

APPROVE:

No violations, MDD within cap, no structurally unfixable conditions.

8. Unit Tests — 79 Tests, All Passing

Tests are written with pytest, using no mocks, no database access, and no LLM calls. Every test uses hand-built numeric fixtures. This is the spec's explicit requirement: 'so every rule is unit-testable with hand-built fixtures, no database access in CI.'

Test file	Tests	What is verified
test_human_capital.py	17	Myers formula correctness; BMS/C&V w_fin bounds and monotonicity; employer concentra
test_constraints.py	18	All four limit constants; strict > vs >=; single/multi-breach detection; all_violations flat list; cle
test_allocation.py	17	Covariance symmetry and PSD; equilibrium formula; FF views shape and structure; BL post
test_risk.py	29	CVaR > VaR ordering; MDD bounds; severity tiers vs drawdown cap; all APPROVE/FLAG/R

Notable test design decisions

Exact-at-limit tests: Every check function uses strict > (not ≥). Tests explicitly verify that a weight of exactly 10% is clean, not a breach. This documents the boundary behaviour for future readers.

15-ticker universe for optimizer tests: With a 10% single-name limit, you need at least 10 tickers for the optimizer to be feasible (10 × 10% = 100%). Tests use 15 tickers to give the optimizer room to work.

BL zero-views test: When views are sent to zero by inflating Ω by 1e10, the BL posterior must converge to the equilibrium prior Π. This validates the BL formula numerically.

9. What Comes Next

Per the spec's build sequencing, the deterministic core is now locked and tested. The remaining work is the LLM wrapper layer:

	File	What it does
	tests/	Adversarial test set for Risk — near-100% rejection on clea
	agents/allocation_agent.py	Thin LLM wrapper: validate AllocationInput, call run_allocat
	agents/risk_agent.py	Thin LLM wrapper: validate AllocationOutput, call run_risk(
	pipeline.py	Orchestration: AllocationAgent → RiskAgent → Compliance

The agents must stay thin. If a number appears in agent code it belongs in `core/`. The LLM's only job in each agent is: (1) parse the user message into a validated schema object, (2) call the relevant core function, (3) turn the output back into readable prose. This boundary is what makes the system auditable.